



# ORACLE

## Academy



# Java Fundamentals

4-4

Chaînes

**ORACLE**  
Academy



# Objectifs

- Voici les objectifs de cette leçon :
  - Instancier (créer) une chaîne
  - Décrire les conséquences de la modification d'une chaîne
  - Utiliser les opérateurs + et += pour concaténer des chaînes
  - Interpréter les séquences d'échappement dans les littéraux de type String



# Présentation

- Cette leçon traite des sujets suivants :
  - Reconnaître la différence entre une chaîne et un type de données char primitif
  - Tester les chaînes avec les méthodes compareTo() et equals()
  - Décrire pourquoi l'opérateur == ne fonctionne pas toujours lorsque l'égalité de chaînes est testée
  - Utiliser les méthodes String length(), substring(), indexOf() et charAt()

# Qu'est-ce qu'une chaîne (String) ?

- Une chaîne est un objet contenant une séquence de caractères
- Les processus de déclaration et d'instanciation de chaîne s'effectuent comme pour toute autre variable d'objet
- Cependant, il existe les différences suivantes :
  - Les chaînes peuvent être instanciées (créées) sans utiliser le mot-clé new
  - Elles ne sont pas mutables
    - Une fois instanciées, elles sont définitives et ne peuvent pas être modifiées



# Modification d'une chaîne

- Vous ne pouvez pas modifier une chaîne Si vous tentez de le faire, un objet String est créé
- En tant que programmeur novice, vous ne verrez pas la différence
- Cependant, c'est un aspect important dans les systèmes réels où le temps de traitement est un élément clé de la conception du programme

# Exemple d'opérations String

```
public class StringOperations{
    public static void main(String[] args){
        String string1 = "Bonjour";
        String string2="Lisa";
        String string3="";           //chaîne vide ou NULL
        string3="Comment ça va ".concat(string2);
        System.out.println("string3: "+ string3);
        //get length
        System.out.println("Longueur: "+ string1.length());
        //get substring beginning with character 0, up to, but
        //not including character 5
        System.out.println("Sous-chaîne: "
                           + string3.substring(0,5));

        //uppercase
        System.out.println("Majuscules: "+string3.toUpperCase());
    } //fin methode main
} //fin classe StringOperations
```

# Modèle de classe

- Utilisez la classe suivante pour insérer des exemples de cette section

```
import java.util.Scanner;  
  
public class StringPractice{  
    public static void main(String[] args){  
        //coller l'exemple de code ici  
    }//fin methode main  
  
}//fin classe StringPractice
```

# Instanciation d'une chaîne

- Les chaînes sont des types de référence d'objet
- Elles peuvent être instanciées de deux façons :
  - Opérateur new
  - Littéraux String
  - Les chaînes ci-dessous ne présentent aucune différence
  - Les deux méthodes d'instanciation créent des objets identiques

```
String s1 = new String("abc"); //opérateur new
String s2 = "abc";             //littéraux String
```

# Références String

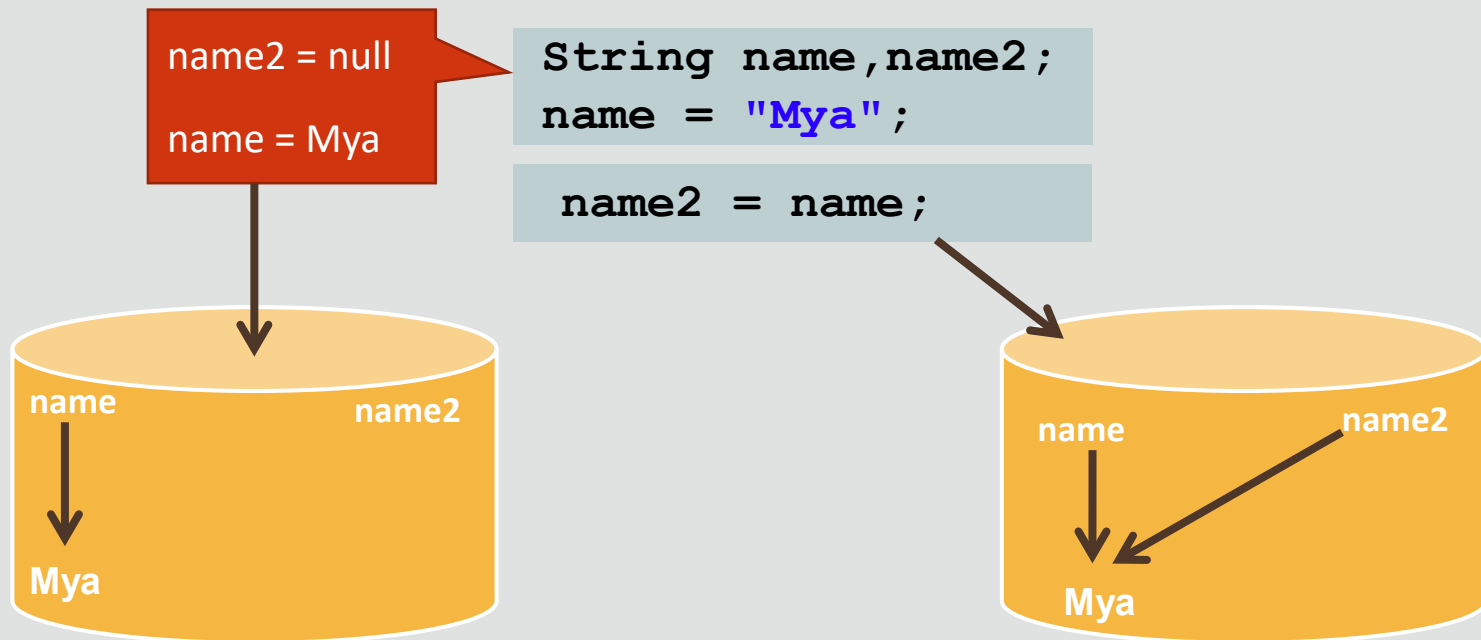
- Lorsque vous créez une référence à un objet ou à une chaîne, l'objet n'existe peut-être pas encore
- Dans le code ci-dessous, étant donné que le nom n'est pas initialisé, le programme ne sera pas compilé
- Le nom de la variable est un pointeur NULL

```
String name;  
System.out.println("Mon nom est " + name);
```

Name = null

# Références String

- Dans ce code, un objet String existe et "name" y fait référence
  - La référence name2 est NULL
  - En quoi cette ligne modifiera-t-elle la mémoire ?

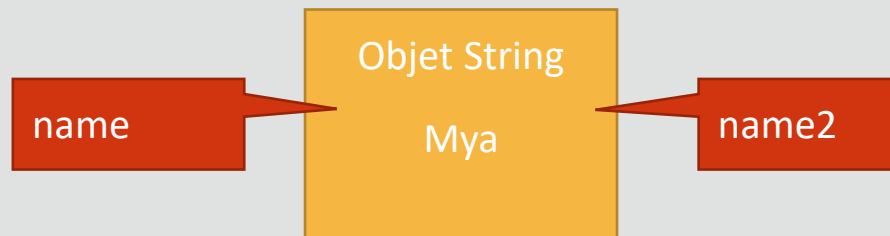


# Références String

- L'exécution de cette ligne de code modifie la référence name2

```
name2 = name;
```

- La référence name2 fera maintenant référence au même objet que name

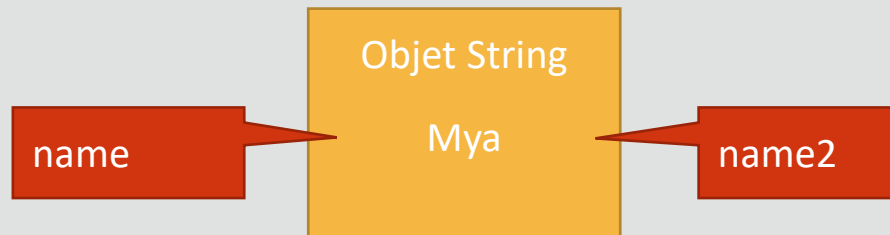


# Références String

- L'exécution de ces lignes de code crée deux instances de la chaîne Mya en mémoire

```
String name, name2;  
name = "Mya";  
name2 = "Mya";
```

- Ici, le compilateur JVM décide d'économiser de l'espace et stocke un seul objet String contenant la chaîne Mya

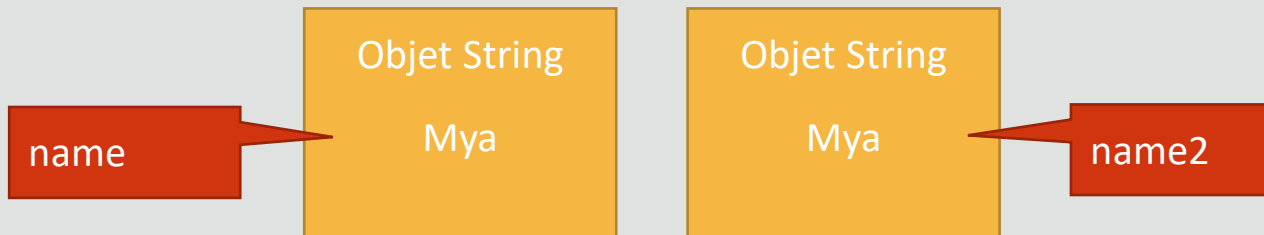


# Références String

- Si nous exécutons un programme très similaire, mais en invitant l'utilisateur à saisir des chaînes, nous obtenons un résultat différent

```
Scanner in= new Scanner(System.in);  
String name, name2;  
name=in.next();  
name2=in.next();
```

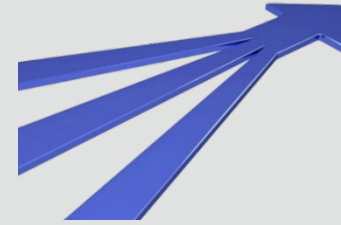
- Si l'utilisateur saisit Mya pour les deux chaînes, le compilateur crée deux objets String différents



# Références String

- Il est très difficile de prévoir si Java va créer une chaîne ou utiliser un objet existant pour une chaîne identique à la nouvelle chaîne





# Concaténation de chaînes

- Concaténez deux chaînes avec les opérateurs + et +=
- Le symbole + permet de concaténer deux chaînes
- Le symbole += permet de concaténer deux chaînes et d'affecter la concaténation en une seule opération

```
String s1 = "Ceci est un ";  
String s2 = "chaîne";  
String s3 = s1 + s2;  
String s4 = "Ceci est un " + s2;  
String s1 += s2;
```

La concaténation est le regroupement de plusieurs éléments dans une chaîne ou une série. La concaténation de chaînes correspond au regroupement d'au moins deux chaînes.



# Manipulation de chaînes

- A chaque fois qu'une chaîne est modifiée, une chaîne est créée à un nouvel emplacement dans la mémoire et la référence est déplacée vers le nouvel emplacement
- Certaines chaînes pointent vers le même objet de référence dans la mémoire
- Si vous modifiez une chaîne qui pointe vers la référence partagée, cette modification n'est pas appliquée aux autres chaînes
- Celle que vous modifiez pointe vers la nouvelle adresse en mémoire qui contient la nouvelle chaîne/la chaîne modifiée, tandis que les autres pointent toujours vers celle d'origine

# Exemple de concaténation de chaînes

- Après l'exécution de ces instructions, quelle sera la sortie pour s1, s2, s3 et s4 ?

```
String s1 = "Ceci est un ";  
String s2 = "chaîne";  
String s3 = s1 + s2;  
String s4 = "Ceci est un " + s2;  
String s1 += s2;  
System.out.println("s1: " + s1);  
System.out.println("s2: " + s2);  
System.out.println("s3: " + s3);  
System.out.println("s4: " + s4);
```

- Entrez ces instructions dans un programme Java et observez les résultats
- Aviez-vous deviné les bonnes réponses ?

# Séquences d'échappement \

- Les séquences d'échappement dans les littéraux String permettent à l'utilisateur d'ajouter des caractères qui seraient mal interprétés par le compilateur
- Par exemple, si vous entrez un guillemet double dans une chaîne, le compilateur l'interprète comme le début ou la fin de la chaîne au lieu de l'y inclure
- Utilisez la séquence d'échappement \" pour inclure un guillemet double

```
String s1 = "Voici un exemple d'échappement.  
\n Nous sommes maintenant sur une nouvelle ligne. \n \t This is a tab."  
String s2 = "\"Ceci est un guillemet \"";  
System.out.println(s1);  
System.out.println(s2);
```

# Séquences d'échappement courantes dans Java

- Voici quelques séquences d'échappement courantes dans Java

| Séquence d'échappement | Représentation         | Représentation alternative d'échappement octal |
|------------------------|------------------------|------------------------------------------------|
| <code>\"</code>        | Guillemet double       | <code>\u0022</code>                            |
| <code>\'</code>        | Guillemet simple       | <code>\u0027</code>                            |
| <code>\\</code>        | Barre oblique inverse  | <code>\u005c</code>                            |
| <code>\t</code>        | Tabulation horizontale | <code>\u0009</code>                            |
| <code>\n</code>        | Retour à la ligne      | <code>\u000a</code>                            |
| <code>\b</code>        | Retour arrière         | <code>\u0008</code>                            |
| <code>\r</code>        | Retour chariot         | <code>\u000d</code>                            |
| <code>\f</code>        | Saut de page           | <code>\u000c</code>                            |
| Echappement octal      | Valeur octale          | <code>\u0000</code> à <code>\u00ff</code>      |

# Méthode compareTo()

- Méthode à utiliser lors de la comparaison de chaînes
  - Méthode : `s1.compareTo(s2)`
  - A utiliser lorsque vous essayez de trouver l'ordre lexicographique de deux chaînes
  - Renvoie un entier
  - Si la chaîne `s1` est inférieure à `s2`, une valeur de type `int`  $< 0$  est renvoyée
  - Si `s1` est égale à `s2`, la valeur `0` est renvoyée
  - Si `s1` est supérieure à `s2`, une valeur de type `int`  $> 0$  est renvoyée



# Méthode equals()

- Méthode à utiliser lors de la comparaison de chaînes
  - Méthode : `s1.equals(s2)`
  - A utiliser lorsque vous souhaitez uniquement savoir si les deux chaînes sont égales
  - Renvoie une valeur booléenne
  - Si la valeur `True` est renvoyée, `s1` est égale à `s2`
  - Si la valeur `False` est renvoyée, `s1` est différente de `s2`



# Méthodes compareTo() et equals() - Exemple

- Qu'affichera cette méthode compareTo() ?

```
String s1 = "abc";  
String s2 = "cde";  
System.out.println(s1.compareTo(s2));
```

- Qu'affichera cette méthode equals() ?

```
String s1 = "abc";  
String s2 = "ABC";  
System.out.println(s1.equals(s2));
```

# Méthodes compareTo() et equals() - Solution de l'exemple

- Cette méthode compareTo affiche une valeur int inférieure à 0

```
String s1 = "abc";  
String s2 = "cde";  
System.out.println(s1.compareTo(s2));
```

- Cette méthode equals affiche "false"

```
String s1 = "abc";  
String s2 = "ABC";  
System.out.println(s1.equals(s2));
```

# Comparaison de chaînes avec l'opérateur ==

- Vous pouvez comparer des variables primitives avec ==
- Cette méthode de comparaison ne fonctionne pas toujours avec les chaînes et doit être évitée, sauf si vous comparez l'adresse en mémoire de deux objets String
- L'opérateur == compare les valeurs de référence, et non les valeurs
- Renvoie true uniquement si les deux objets String pointent vers le même objet de référence

```
String s1 = "Ceci est un chaîne.";
String s2 = new String("Ceci est un chaîne.");
String s3 = "chaîne.";
String s4 = "Ceci est un " + s3;
System.out.println(s1 == s2);
System.out.println(s1 == s4);
```

# Comparaison de chaînes avec l'opérateur ==

- Dans l'exemple ci-dessous, quelle partie du code affiche true ?
  - s1 et s2 pointent vers le même objet
  - Java remarque que ces chaînes sont identiques et en déduit une seule référence
  - s5 ne pointe pas vers le même objet que s1
  - Elle est "créée" à partir d'une combinaison de deux chaînes, donc un objet est créé

```
String s1 = "Ceci est un chaîne.";
String s2 = "Ceci est un chaîne.";
String s3 = "Ceci est ";
String s4 = "un chaîne.";
String s5 = s3 + s4;
System.out.println(s1 == s2);
System.out.println(s1 == s5);
```

# Méthode String utile : length()

- Méthode : s1.length()
- Renvoie la longueur (nombre de caractères) de s1 sous forme de valeur int
- Il s'agit d'une méthode d'accès appelée sur un objet String qui renvoie la variable de longueur de la chaîne

```
String s1 = "Ceci est un chaîne." ;  
int n = s1.length() ;  
//n est égal à 16 car s1 a 16 caractères
```

# Chaque caractère d'une chaîne a un index

- Etant donné que les chaînes sont une représentation d'une séquence de caractères, tous les caractères d'une chaîne ont un index
- Un index est une référence de position
- Le premier caractère d'une chaîne a un index de 0, le deuxième a un index de 1, et ainsi de suite jusqu'à la fin de la chaîne



# Méthode String utile : substring()

- `s2.substring(int beginIndex)`
  - Renvoie une partie de la chaîne `s2` de l'index `beginIndex` à la fin de la chaîne
- `s2.substring(int beginIndex, int endIndex)`
  - Renvoie une partie de la chaîne `s2` de l'index `beginIndex` à `endIndex`, mais ne comprend pas le caractère à l'index `endIndex`
  - Quel est le résultat de l'exécution de ce segment de code ?

```
String s1 = "Je mange des pommes";  
String s2 = "La banane est mon fruit préféré";  
System.out.println(s1.substring(6)  
                    +" "+s2.substring(8,23));
```

# Méthodes String utiles : indexOf et charAt

- Méthode : **s3.indexOf(char c);**
  - Renvoie la valeur d'index de la première occurrence de la lettre c dans la chaîne s3
- Le paramètre ne doit pas forcément être un caractère, il peut aussi être une chaîne, par exemple :  
**s3.indexOf("le");**
- Méthode : **s4.charAt(int index);**
  - Renvoie le caractère de la chaîne situé à l'index transmis en tant que paramètre
  - L'index peut être un entier compris entre 0 et **s4.length()-1**

# Utilisation des méthodes String - Exemple 1

- Ecrivez le code Java de sorte qu'il prenne une adresse électronique en tant qu'entrée sous la forme d'une chaîne Cette méthode renvoie le domaine
  - Par exemple, si l'utilisateur saisit john@oracle.com, la méthode renvoie oracle.com
  - Pour résoudre ce problème, recherchez d'abord le caractère "@" dans la chaîne
  - Ensuite, obtenez la portion de sous-chaîne de la chaîne initiale qui vient après cette position

```
String domain=""; //aucun espace entre les deux "  
int position=email.indexOf('@');  
domain=email.substring(position+1);  
System.out.println(domain);
```

# Terminologie

- Termes clés utilisés dans cette leçon :
  - Concaténation
  - Séquences d'échappement
  - Instancier
  - Objet de référence
  - Objet String
  - Méthodes String `compareTo()` et `equals()`

# Synthèse

- Dans cette leçon, vous avez appris comment :
  - Instancier (créer) une chaîne
  - Décrire les conséquences de la modification d'une chaîne
  - Utiliser les opérateurs + et += pour concaténer des chaînes
  - Interpréter les séquences d'échappement dans les littéraux de type String
  - Reconnaître la différence entre une chaîne et un type de données char primitif
  - Tester les chaînes avec les méthodes compareTo() et equals()
  - Décrire pourquoi l'opérateur == ne fonctionne pas toujours lorsque l'égalité de chaînes est testée
  - Utiliser les méthodes String length(), substring(), indexOf() et charAt()





# ORACLE

## Academy

